

BOHAN-PETA

Product & System Requirements Document (PRD / SRS)

Digital quiz platform for course cohorts (QA and other courses)

Version 2.4 | Engineering handoff draft — written for implementation with Claude Code

Date: August 12, 2026

Platform: Progressive Web App (PWA), mobile-first, installable on iOS + Android

Table of Contents

Table of Contents.....	2
0. Decisions Locked In For This Version	3
1. Introduction & Goals	3
1.1 Business Goals	4
1.2 Out of Scope (v1.0)	4
2. Users & Roles	4
3. Core User Flows	4
3.1 Student login and quiz-taking	4
3.2 Teacher: creating a quiz — AI + manual review.....	6
3.3 Teacher: assigning a quiz to a cohort and reviewing scores	7
3.4 Quiz lifecycle: Edit Mode ⇌ Execution Mode	7
3.5 Per-question AI improvement dialog	7
3.6 Quiz-level feedback (as distinct from per-question feedback)	8
3.7 Time limit, pass score, and pass/fail feedback	8
3.8 Focus lock during the exam (anti-cheating deterrent)	8
4. Functional Requirements	9
4.1 Student module	9
4.2 Teacher/Admin module — quiz creation.....	10
4.3 Teacher/Admin module — cohorts & assignment	11
5. Data Model	11
5.1 Relationships.....	12
5.2 Quiz import/export format	12
5.3 Scores per cohort (Jan 2026 / Mar 2026, etc.)	13
5.4 Quiz lifecycle and score integrity	13
6. AI-Assisted Question Generation	13
6.1 Process	13
6.2 Key considerations.....	14
7. Non-Functional Requirements	14
8. Recommended Technical Architecture	14
9. Implementation Appendix (for engineering handoff)	15
9.1 Suggested repository layout.....	15
9.2 Database schema (PostgreSQL DDL, starting point)	15
9.3 REST API surface (starting point).....	17
9.4 Suggested build order for Claude Code.....	19
9.5 Sample quiz — worked example / seed fixture	19
10. Risks & Open Questions	20
11. Success Metrics	20

0. Decisions Locked In For This Version

v2.4 change: three corrections to 3.1. (1) Access model: one shared Join Exam URL for every exam, distinguished only by a token the student types in — not a unique per-assignment link as earlier drafts had it (F-22 updated accordingly). (2) A required per-quiz time limit (duration_minutes), enforced server-side, that auto-submits and grades on expiry (F-04, new 'time_expired' ended_reason). (3) Paginated, freely-navigable exam UI — one question per page, answers editable at any point, Submit button on the last page (F-04a/b/c).

v2.3 change: added a post-submission review link (F-07a/F-07b, 3.1) so a student can immediately see which questions they got wrong. This resolves what was an open product question in v2.2 as two independent settings: per-question correct/incorrect status (default on, low leak-risk) and full answer-key reveal (default off, higher leak-risk) — see 3.1 for the reasoning.

v2.2 change: the AI generation engine recommendation switched from Claude API to the Google Gemini API, per owner decision — see the updated row in section 8 and the design note directly beneath it on keeping the provider swappable.

v2.1 change: the quiz lifecycle was previously modeled as a one-way "draft → published" flow. This version corrects that — editing must always be reachable, even after a quiz has gone live. See the new section 3.4 and the schema/API updates in sections 5 and 9. This was a real gap in v2.0, not a rewording.

This version otherwise supersedes v1.0. Two open questions from the earlier draft have been resolved as follows, per product-owner sign-off:

Topic	Decision	Rationale
Client platform	Progressive Web App (PWA), not a native iOS/Android app	Nothing in scope requires native-only capability (push notifications, full offline exam-taking). A PWA ships from one codebase, installs to the home screen on both platforms, and can be wrapped with Capacitor into native shells later without a rewrite, if a real native-only need appears.
Score storage per cohort	Single relational schema; a Cohort field scopes every quiz assignment and every score record. No duplicated per-cohort tables.	Keeps cross-cohort reporting, quiz reuse, and schema maintenance simple, while the UI and CSV/Excel export still present each cohort as its own isolated sheet.
Product name	BOHAN-PETA (previously "Quizit")	Owner decision.
Document language / audience	English; written as an engineering brief intended to seed implementation work in Claude Code	Previous version targeted stakeholders; this version adds a technical appendix (schema DDL, API surface, repo layout) so it can be handed directly to an engineering agent.
AI generation engine	Google Gemini API (previously Claude API)	Owner already holds a paid Gemini subscription, making it the lower-marginal-cost option. See section 8 for the full rationale and the recommendation to keep the provider swappable behind an internal interface.

1. Introduction & Goals

BOHAN-PETA is a digital quiz platform for training institutions and professional courses (initial use case: QA / software testing courses, built generically enough to support any course). Teachers create quizzes — with AI assistance from source documents — and assign them to one or more cohorts (course runs). Students take the quiz on their phone via a web link and receive an instant score.

1.1 Business Goals

- Cut the time teachers spend building assessments, via AI-assisted question generation from existing course material (e.g. official docs such as playwright.dev).
- Allow one quiz to be reused across multiple cohorts of the same course, while keeping each cohort's scores fully isolated.
- Give students a fast, simple mobile exam experience with an immediate score.
- Give teachers a clear view of performance per cohort and per student.

1.2 Out of Scope (v1.0)

- Payments / billing.
- Open-ended / essay questions — only Multiple Choice / Multiple Select.
- Integration with an external LMS (Moodle, Google Classroom, etc.) — noted as a future extension.
- Full offline exam-taking (see Non-Functional Requirements for the resilience level actually targeted).

2. Users & Roles

Role	Description	Core capabilities
Student	Joins a specific quiz via a shared Join Exam URL plus a token from their teacher (3.1)	One-time registration per attempt, take the quiz, view only their own score
Teacher / Course Admin	Owns quiz creation, cohort management, and score tracking	Create quizzes (AI + manual), manage cohorts, assign quizzes, view and export scores
Super Admin (future)	Cross-organization admin, multiple teachers	Manage teacher accounts, permissions, global settings

Scope note v1.0 should ship with exactly two roles (Student, Teacher/Admin). Defer Super Admin / multi-teacher organizations to a later phase unless there is already more than one teacher using the system today.

3. Core User Flows

3.1 Student login and quiz-taking

1. Student navigates to the app's Join Exam screen — a single URL, the same for every exam the platform ever runs. There is no per-assignment link.
2. Student enters: first name (required), last name (required), national ID (required, format-validated), email (optional — for result delivery/updates), and the exam token — a short code given by the teacher (e.g. 123456) that identifies this specific quiz assignment (this quiz, for this cohort).
3. System looks up the quiz assignment by that token, and checks whether this national ID already has an attempt on it (see the single-attempt rule in 5.3).
4. On success, the student proceeds to the quiz: one question per page, page number = question number (question 5 is page 5), 3–20 questions per the teacher's configuration. A countdown for the quiz's time limit (below) starts running.

5. The student can move freely between pages via a question navigator (e.g. numbered 1..N) — not just forward — and can change an already-given answer on any page at any point before submitting. Answers are saved as the student navigates, not only at the end (this is also what powers resuming after a disconnect, F-05).
6. The "Submit Exam" button appears on the last question's page.
7. If the time limit is reached before the student submits — see below — the attempt is auto-submitted immediately using only the answers given so far.
8. If the exam screen loses focus at any point during the attempt (student switches app, tab, or window) — see 3.8 — the attempt is auto-submitted immediately using only the answers given so far.
9. On submit (manual, time-expiry, or focus-loss auto-submit), the system computes a score (0–100%) against the quiz's pass score, and shows a result screen: pass/fail feedback text as configured by the teacher (3.7), a link to review the exam (below), and an option to return to the home screen.
10. The review link opens a per-question breakdown of that specific attempt: every question the student was asked, with their own answer marked correct or incorrect — so the student can immediately see which questions they got wrong, without necessarily being told what the right answer was (see the two-tier setting below). If an email was provided, the same link is included in the confirmation email (F-07) so the student can return to it later.

Design note — token entry vs. per-assignment links

This is a deliberate access model, not just an implementation detail: every exam is reached through the same Join Exam URL, and the token typed in is what selects the quiz assignment — there is no unique link per cohort/quiz to distribute or manage. This matches how a teacher would actually hand out access in a classroom (read a short code aloud, write it on a board), and it's a different mechanism from the review link (3.1, below) and the deep-link style referenced in earlier drafts, which was per-assignment. Tokens should be short and easy to communicate (e.g. 6 digits), unlike the Attempt UUID used for the review link, which is deliberately long and unguessable because it's a bearer secret with no separate identity check in front of it — the token, by contrast, is always paired with the student's own name/ID entry, so it doesn't need to carry the same unguessability burden on its own.

This closes what was an open product question in earlier drafts, now resolved as two independent, per-quiz teacher settings, since they carry very different leak risk:

- Per-question correct/incorrect status (default ON): the review link always shows, for each question, whether the student's own submitted answer was right or wrong. This reveals nothing about what the correct option actually was for a question the student got wrong, so it carries low risk of leaking answers to later test-takers, and directly serves the goal of letting a student learn from their own mistakes right away.
- Full answer key reveal (default OFF, separate teacher toggle): additionally shows the correct option(s) for every question, including ones the student answered incorrectly. This is the setting worth defaulting off, since it's what could let a student pass on exact correct answers to someone taking the same quiz later — the per-question status above does not have that problem.
- All three settings — time limit, focus-loss auto-submit, and the review tiers — apply identically regardless of how the exam ended (manual submit, time expiry, or focus loss): none of them unlocks a different result than a normal submission would have gotten.

Design note — the review link is a bearer secret

Because students don't have accounts or passwords beyond the one-time exam token, the review link itself is what proves it's that student's attempt — anyone holding the link can open it. It must therefore be built as an unguessable token (the Attempt UUID is sufficient, but should never be a short or sequential ID), and the review endpoint must not expose any way to enumerate or guess other students' attempts. This is a different secret from the exam token above — the exam token is short and shared knowledge for a whole cohort, paired with the student's own identity fields; the review link is per-attempt and must stand on its own.

Design note — server-side time enforcement

Unlike the focus-loss check (3.8), which is fundamentally a client-reported signal with real limits, the time limit can and should be enforced authoritatively on the server: reject or force-submit any answer-saving request once `now()` exceeds `Attempt.started_at + QuizTemplate.duration_minutes`, regardless of what the client's own countdown UI shows or whether the client-side timer was tampered with or paused. The client countdown is a display convenience for the student; the actual cutoff is a server-side timestamp comparison.

Recommend also running a small server-side sweep (or a check on next request) to auto-submit attempts that were left open past their duration without the client ever calling submit.

3.2 Teacher: creating a quiz — AI + manual review

1. Teacher opens the admin panel and selects "New Quiz".
2. Teacher provides one or more content sources: any mix of document URLs (e.g. <https://playwright.dev/python/docs/>) and uploaded files (PDF/DOCX/plain text). Multiple sources are combined into one quiz.
3. Teacher sets generation parameters — see the table below.
4. System calls the AI generation service (see section 6), which returns a draft: question text, answer options, the correct option(s) flagged, a reference back to the source passage, and — where requested — an image or diagram per question.
5. Teacher reviews the draft in an edit screen: can edit question/answer wording, delete a question, request "regenerate this question", add a fully manual question, and change which option is marked correct.
6. Teacher saves and publishes (moves the quiz template from Edit Mode to Execution Mode). The quiz is stored as a Quiz Template — an entity independent of any cohort, assignable later to multiple cohorts. Publishing is not a one-way door: see 3.4.

Parameter	Description
Language	The language the quiz is generated and displayed in (e.g. Hebrew, English). Independent of the source document's language — the teacher may generate a Hebrew quiz from English docs, or vice versa.
Difficulty	A single level for the whole quiz (e.g. Easy / Medium / Hard), used to steer question complexity, distractor plausibility, and depth of the concepts tested.
Number of questions	3–20, per F-11.
Sources	One or more URLs and/or uploaded files, combined as the material the AI draws questions from (3.2, step 2).
Images / diagrams per question	Teacher can request an illustrative image or diagram for a question — either for the whole quiz ("add visuals where useful") or toggled per question after the draft is generated. See the design note below on how this is actually produced.
Sub-topics (optional)	Free-text emphasis, e.g. "focus on Locators and Network Interception".
Answer options per question	2–6, per F-11.
Multiple correct answers allowed	Whether the generator may produce Multiple Select questions in addition to Multiple Choice.

Design note — images and diagrams (F-10c)

Two distinct capabilities are being bundled under "add an image", and they have very different reliability profiles: (1) pulling an existing image directly from the source document (e.g. a screenshot already in the Playwright docs) is low-risk and should be preferred whenever the source contains one relevant to the question; (2) generating a new diagram or illustration from scratch is higher-risk for technical/factual questions — an AI-drawn architecture diagram can be subtly wrong in a way that's hard for a teacher to spot-check as quickly as wrong text would be. Recommend scoping v1 to (1) plus simple, structurally-generated diagrams (e.g. flowcharts rendered from an explicit step list the AI proposes, which a teacher can read and verify as text before it's rendered) rather than open-ended image generation. Full illustrative image generation can be a v2 add-on once there's a review step the teacher trusts.

Design note

Quiz Template (the questions themselves) and Quiz Assignment (when, to which cohort, until when, how many attempts) must be modeled as separate entities. This is what makes "one quiz, many cohorts" possible — see the data model in section 5.

3.3 Teacher: assigning a quiz to a cohort and reviewing scores

- 1. Teacher creates/selects a Cohort (e.g. "QA January 2026").
- 2. Teacher assigns an existing quiz template to that cohort, and sets: open/close dates, max attempts per student, whether to shuffle question/answer order.
- 3. System generates a unique link/code for this assignment, to distribute to the cohort's students.
- 4. Once students have completed the quiz, the teacher opens the "Scores" view scoped to that specific cohort — a table of student vs. score, submission time, and time taken.
- 5. Teacher can export that cohort's score sheet alone to Excel/CSV.

3.4 Quiz lifecycle: Edit Mode ⇌ Execution Mode

This is a hard requirement, not an implementation detail: a teacher must always be able to return to editing a quiz — to fix wording, delete or add questions manually, ask the AI to rephrase or improve a question, or ask the AI to extend the quiz with new questions — even after that quiz has been published and used live. Edit access is never revoked by publishing.

State	What the teacher can do	What students see
Edit Mode	Add, delete, or reorder questions; edit question/answer text manually; request AI rewrite or improvement of a question; request AI to generate additional questions; change which option is correct	Nothing — a quiz in Edit Mode cannot be joined by students, even if it has been published before
Execution Mode (published)	Everything above is still available by switching back to Edit Mode — publishing does not lock the quiz	Can join and take the quiz via an active Quiz Assignment

- The transition Execution → Edit is always available. The transition Edit → Execution ("publish") requires the quiz to have at least the minimum question count (3) and no unresolved AI-draft questions.
- Switching back to Edit Mode does not delete or invalidate existing Quiz Assignments or previously submitted Attempts — a cohort that already finished the quiz keeps its recorded scores regardless of later edits.
- If a Quiz Assignment is still open (students are actively able to join) when the teacher edits the underlying template, the edited question set applies to anyone who joins from that point on. Students already mid-attempt keep the question set they started with.

Design consequence worth deciding on

Because a Quiz Assignment references the live Quiz Template rather than a frozen copy, editing a template that is currently assigned to an open cohort changes what new joiners see, mid-window. If you want a cohort's exam to be provably identical for every student regardless of later improvements to the master quiz, the answer is not to lock editing — it's to use "Duplicate quiz" (F-15) before assigning it to that cohort, and keep iterating on the original template separately. Recommend surfacing this as a one-line hint in the UI when a teacher edits a template that has an open assignment.

3.5 Per-question AI improvement dialog

In Edit Mode, every single question — whether AI-generated or manually written, published before or not — has its own "Improve with AI" action, opening a small dialog scoped to that one question only:

- 1. Teacher clicks "Improve with AI" on a specific question row.

2. A dialog opens showing the current question text and options, plus a free-text "Notes for AI" field — e.g. "make this less ambiguous", "this is too easy, raise the difficulty", "the distractors are too obviously wrong", "rewrite using the terminology from section 4 of the source doc", or "this question is redundant with question 4 — propose a replacement".
3. Teacher submits the dialog. The system sends the current question, its options, its source_reference, and the teacher's notes to the AI, and returns a proposed replacement.
4. The proposed replacement is shown as a diff/preview against the current question — teacher can Accept (replaces the question in place), Discard (keeps the original, notes are still saved for next time), or edit the AI's proposal manually before accepting. If the note was a redundancy/removal flag, Accept can also mean simply deleting the question rather than replacing it.
5. The notes the teacher entered are saved on the question (not just used once) so they carry over as context the next time "Improve with AI" is opened on that same question — the teacher doesn't have to retype standing instructions like "keep answers under 15 words" every time.

This dialog is the mechanism behind F-19, and is available identically whether the quiz is currently in Edit Mode or was just switched back to Edit Mode from Execution Mode (3.4) — there is no separate "republish" step required to keep using it.

3.6 Quiz-level feedback (as distinct from per-question feedback)

Section 3.5 covers feedback scoped to one question. Separately, the quiz review screen has a general "Notes on this quiz" field, scoped to the whole quiz — e.g. "this is too hard overall, ease it up", "too much emphasis on tracing, add more on locators", "question 2 is redundant with question 5". This is a distinct mechanism because a whole-quiz note often implies changes spanning multiple questions at once, not a single-question edit.

1. Teacher enters general notes in the quiz-level feedback field and submits.
2. System sends the full current question set plus the teacher's notes to the AI, which proposes a batch of changes — this may include rewording several questions, replacing weak ones, adjusting overall difficulty, or removing a question flagged as redundant.
3. Teacher reviews the proposed batch as a set of per-question diffs (same review UI pattern as 3.5, just pre-populated across several questions at once) and accepts/edits/discards each one individually before saving.
4. Quiz-level notes are also persisted on the quiz template, the same way per-question notes persist on a question (F-19a), so standing instructions like "keep this quiz at medium difficulty" carry forward.

A quiz-level note that references a specific question by number ("question 2 is redundant") is still routed through this same general field — the system does not require the teacher to separately open that question's own dialog (3.5) for that kind of feedback, though they may do either.

3.7 Time limit, pass score, and pass/fail feedback

Every quiz template carries three grading-related settings, all editable at any time — including after the quiz has been published and used by students (per the Edit Mode $\rightleftharpoons$ Execution Mode rule in 3.4):

- A maximum answer time in minutes (e.g. up to 30 minutes), starting when the student begins the attempt. Once it elapses, the attempt is auto-submitted using whatever answers were given up to that point — the exam closes and a score is issued based on progress at that moment, exactly as described in 3.1. This is a required setting per quiz template, not optional, since every quiz needs some bound even if generous.
- A pass score (0–100). One quiz can require 60% to pass while a completely different quiz, with different questions, requires 75% — these are independent settings on independent templates. Changing a quiz's pass score does not retroactively change any already-computed Attempt.score values (scores are stored as a raw percentage, per 5.4) — but it does change how those same scores are classified as pass/fail going forward, including for the pass/fail feedback message shown on the result screen and in any score export.
- Exactly two feedback messages: one shown to a student whose score is at or above the pass score ("pass feedback"), and one shown to a student whose score is below it ("fail feedback"). No support for more than these two tiers is needed at this time — e.g. no separate "distinction" tier. These are plain free-text messages configured by the teacher (not AI-authored), e.g. "Well done — you're ready to move on to the next module" vs. "You'll need to review the Locators section before retaking this material."

3.8 Focus lock during the exam (anti-cheating deterrent)

While a student has an exam in progress, the exam screen must always stay in focus. If the student navigates away — switches to another app, another browser tab, minimizes the window, or the device locks — the attempt ends immediately: it is auto-submitted using whatever answers were given up to that point, and the student sees the normal result screen (3.7), just as if they had pressed Submit. This is intended to discourage switching away to look something up or ask someone else during the exam.

Design note — what this can and can't guarantee on a PWA

This needs to be understood as a strong deterrent, not a hard technical guarantee — and that limitation follows directly from the platform decision in section 0. In a browser/PWA, the only signals available are the Page Visibility API (visibilitychange) and window blur/focus events: reliable for the common cases (switching apps, switching tabs, locking the phone), but not something the app can make unbypassable — a sufficiently determined student on a second device, or using OS-level split-screen in a way that doesn't always fire these events, is not stopped by this mechanism, only slowed down and discouraged. A genuinely lockable exam session (preventing app-switching at the OS level) generally requires native platform APIs — iOS Guided Access, Android Screen Pinning — which an installed native app can prompt the user into, but a web page cannot invoke at all. This is the concrete, specific case where the native-vs-PWA tradeoff from section 0 has real teeth: if strict lockdown is a hard requirement rather than a deterrent, that pushes toward the native-wrapper path noted as a reversibility option there, rather than something a PWA can be made to do. Recommend shipping the PWA version first, since it meaningfully raises the bar over having no check at all, while treating true lockdown as a distinct future ask if it turns out to matter in practice. Note the contrast with the time limit in 3.7, which — unlike this focus check — can and should be enforced authoritatively server-side, not just as a client signal.

- Scoring on auto-submit: the score denominator is always the full question count, so unanswered questions (including all questions after the point of exit) count against the score exactly as they would if the student had left them blank and pressed Submit manually.
- The student is shown a clear, neutral message that the exam ended because they left the exam screen, followed by the normal pass/fail result (3.7) — not a specialized or harsher message, to avoid ambiguity about whether this was a bug versus an intentional rule.
- This check only applies while an attempt is actively in progress — it has no effect on the Join Exam screen, the result screen, or any teacher-side screens.

4. Functional Requirements

4.1 Student module

#	Requirement	Priority
F-01	Capture student details: first name, last name, national ID (required), email (optional), exam token	Must
F-02	Validate national ID format (configurable per country; default Israeli ID: 9 digits + check digit)	Must
F-03	Prevent a duplicate attempt on the same quiz assignment by national ID, unless the teacher enabled multiple attempts	Must
F-04	Required maximum answer time per quiz (minutes), enforced server-side; auto-submit and grade using answers given so far once it elapses (3.7)	Must
F-04a	Paginated exam UI: one question per page, page number equals question number (question 5 = page 5)	Must
F-04b	Free navigation between question pages (not just linear forward), with previously given answers editable at any point before submit	Must
F-04c	"Submit Exam" button appears on the last question's page	Must
F-05	Resume an in-progress attempt after an accidental disconnect (draft state), preserving answers already saved while navigating (F-04b)	Should

#	Requirement	Priority
F-06	Show the final score immediately on submission, plus the teacher-configured pass/fail feedback message (3.7)	Must
F-07	Email the score/confirmation to the student, if an email was provided, including the review link (F-07a)	Should
F-07a	Provide a per-attempt review link showing each question with the student's own answer marked correct/incorrect — default-on, low leak-risk (3.1)	Must
F-07b	Per-quiz teacher toggle to additionally reveal the correct option(s) on the review link, even for questions the student got wrong — default off (3.1)	Should
F-08	Detect loss of focus/visibility on the exam screen during an active attempt (app switch, tab switch, window minimize, device lock) and immediately auto-submit using answers given so far (3.8)	Must
F-08a	On auto-submit due to focus loss, show the same result screen a manual submit would show — clearly stating the exam ended because the screen lost focus	Must

4.2 Teacher/Admin module — quiz creation

#	Requirement	Priority
F-10	Create a quiz from one or more source URLs and/or uploaded files (PDF/DOCX/TXT), combined into a single generation request (3.2)	Must
F-10a	Select the quiz's output language independently of the source document's language	Must
F-10b	Select a single difficulty level for the whole quiz (Easy / Medium / Hard) to steer generation	Must
F-10c	Request an image or diagram be attached to a question — prefer sourcing an existing image from the source document; AI-generated diagrams limited to structurally-verifiable content (see design note in 3.2)	Should
F-11	Configure question count (3–20) and number of answer options per question	Must
F-12	AI-generated draft questions with mandatory manual teacher review/approval before publishing	Must
F-13	Support Multiple Choice (single correct answer) and Multiple Select (several correct answers)	Must
F-14	Support fully manual question authoring, with no AI involved	Must
F-15	Duplicate an existing quiz as the basis for a new one	Should
F-16	Import/export a question bank in a defined file format (see 5.4)	Must
F-17	Shared question bank for reuse across quizzes	Should
F-18	Return a published (Execution Mode) quiz to Edit Mode at any time, without affecting existing assignments or already-submitted attempts	Must
F-19	Per-question "Improve with AI" dialog: teacher writes free-text notes for one specific question, AI proposes a revision, teacher accepts/edits/discards (see 3.5)	Must
F-19a	Persist the teacher's per-question AI notes on the question itself, so they carry over as context for future "Improve with AI" requests on that question	Should

#	Requirement	Priority
F-19b	Quiz-level "Notes on this quiz" feedback (e.g. "too hard overall", "question 2 is redundant") that proposes a batch of changes across multiple questions at once (3.6)	Must
F-27	Set a pass score (0–100) per quiz template, editable at any time including after publishing (3.7, 3.4)	Must
F-28	Configure exactly two teacher-authored feedback messages per quiz template: shown on pass, shown on fail (3.7)	Must

4.3 Teacher/Admin module — cohorts & assignment

#	Requirement	Priority
F-20	Create/edit/archive a Cohort (course run) with a name and start/end dates	Must
F-21	Assign an existing quiz template to multiple cohorts in parallel, each assignment with independent settings	Must
F-22	Generate a unique access token per quiz-cohort assignment, entered by the student on the single shared Join Exam screen (3.1) — not a per-assignment link	Must
F-23	View a cohort-scoped score sheet, with sort and filter	Must
F-24	Export a single cohort's score sheet to Excel/CSV	Must
F-25	Cross-cohort comparison report (average score, pass rate) — future	Should (V2)
F-26	Void/disable a specific student or attempt (for errors or suspected cheating)	Should

5. Data Model

A single relational schema. Cohort isolation is achieved via a foreign key and scoped queries, not via duplicated tables — this is what lets one quiz template serve many cohorts, while both the UI and CSV/Excel export still present each cohort as an isolated sheet.

Entity	Key fields	Notes
Student	id, first_name, last_name, national_id, email (nullable)	Identified by national_id + name; no personal login — access is via the per-assignment exam token entered on the shared Join Exam screen (3.1)
Teacher/Admin	id, name, email, password_hash, role	Standard username/password auth
Cohort	id, name ("QA January 2026"), start_date, end_date, teacher_id	Represents one course run
QuizTemplate	id, title, language, difficulty, created_by, created_at, question_count, ai_generated (bool), status (draft/published), teacher_notes, duration_minutes, pass_score, pass_feedback_text, fail_feedback_text, reveal_answer_key (bool)	The questions themselves — independent of any cohort. status toggles Edit Mode ⇌ Execution Mode (3.4); editing is allowed in either state. teacher_notes holds standing quiz-level AI feedback (3.6, F-19b). duration_minutes/pass_score/feedback fields per F-04/F-27/F-28 (3.7). reveal_answer_key gates the full-answer-key tier of the review link, default false (3.1, F-07b)

Entity	Key fields	Notes
QuizSource	id, quiz_template_id, source_type (url/file), url, storage_key	One or more sources per quiz template (F-10) — replaces a single source_document_url field
Question	id, quiz_template_id, text, type (single/multi), source_reference, is_active (bool), teacher_notes, image_url, image_prompt	Belongs to a quiz template. Soft-deleted (is_active=false) rather than hard-deleted once it has recorded attempt answers, to preserve historical attempt data. teacher_notes holds standing per-question AI instructions (3.5, F-19a). image_url/image_prompt support F-10c
AnswerOption	id, question_id, text, is_correct (bool)	2–6 options per question
QuizAssignment	id, quiz_template_id, cohort_id, access_code, open_at, close_at, max_attempts, shuffle (bool)	Links a quiz template to a cohort, with per-assignment settings. access_code is the short exam token students type on the shared Join Exam screen (3.1) — not part of a per-assignment URL
Attempt	id, quiz_assignment_id, student_id, started_at, submitted_at, score, ended_reason (submitted/focus_loss/time_expired)	The actual score record. ended_reason distinguishes a normal submit from an auto-submit triggered by focus loss (3.8, F-08) or by the quiz's time limit elapsing (3.7, F-04)
AttemptAnswer	id, attempt_id, question_id, selected_option_ids[]	For audit/analysis

5.1 Relationships

- Teacher 1—N Cohort
- Teacher 1—N QuizTemplate
- QuizTemplate 1—N QuizSource
- QuizTemplate 1—N Question 1—N AnswerOption
- QuizTemplate 1—N QuizAssignment N—1 Cohort (this is what makes one quiz ⇔ many cohorts possible)
- QuizAssignment 1—N Attempt N—1 Student

5.2 Quiz import/export format

Two supported formats:

- JSON as the source of truth for a quiz template — questions, options, correct-answer flags, metadata (source, difficulty). This is what the app itself persists, and what's used for sharing quizzes between teachers or backing up versions.
- Excel/CSV as a human-editable format — one row per question, one column per answer option, plus a "correct answer" column. Useful for a teacher pre-authoring questions in a spreadsheet, or exporting for offline review.

Proposed JSON shape:

```
{
  "quiz_title": "Playwright Python Basics",
  "language": "en",
  "difficulty": "medium",
  "sources": [
    "https://playwright.dev/python/docs/"
  ],
  "duration_minutes": 30,
  "pass_score": 60,
}
```

```

"pass_feedback_text": "...",
"fail_feedback_text": "...",
"reveal_answer_key": false,
"questions": [
  {
    "text": "...",
    "type": "single",
    "source_reference": "...",
    "image_url": null,
    "options": [
      {"text": "...", "is_correct": true},
      {"text": "...", "is_correct": false}
    ]
  }
]
}

```

image_url is null unless an image/diagram was requested and produced for that question (F-10c); sources is always an array, even for a single source, to support F-10's multi-source generation; pass_score/feedback fields per F-27/F-28 (3.7); reveal_answer_key per F-07b (3.1) — the per-question correct/incorrect status itself (F-07a) is not a quiz-level flag, since it's always on.

See 9.5 for a full worked example with real content.

Both directions (JSON $\rightleftharpoons$ Excel) should be supported so teachers can use whichever tool they prefer.

5.3 Scores per cohort (Jan 2026 / Mar 2026, etc.)

- Every attempt is linked to a QuizAssignment, which is linked to a specific Cohort. Assigning the same quiz template to "QA January 2026" and "QA March 2026" produces two completely separate assignment records — and therefore two fully isolated score pools — even though the underlying questions are identical.
- The teacher's Scores screen always starts with a cohort selector, so in practice the experience is exactly like a separate sheet per cohort, with no schema duplication behind it.
- Export is always scoped to the selected cohort, so a teacher who wants literal separate Excel files per cohort still gets that, on demand.
- A future (V2) cross-cohort report can be added purely as a new query — no data model change required.
- This is what satisfies the requirement that one exam used by several cohorts, even sat on different dates, keeps every cohort's scores grouped and separate: the grouping key is always QuizAssignment ($\rightarrow$ Cohort), never the calendar date an individual student happened to take it.

5.4 Quiz lifecycle and score integrity

- QuizTemplate.status ('draft' | 'published') controls whether a quiz can be joined by students. It does not control whether it can be edited — editing is always allowed, per section 3.4.
- Attempt.score is computed and persisted at submission time, not recalculated later — so editing a template after a cohort has already submitted attempts never changes a stored score.
- Question rows are never hard-deleted once at least one AttemptAnswer references them; instead Question.is_active is set to false so the question drops out of future assignments while historical attempt data stays intact and queryable.

6. AI-Assisted Question Generation

6.1 Process

1. Fetch content from the source (URL scraping / uploaded-file parsing).
2. Chunk the content into study-relevant segments.
3. Call the AI generation engine (Google Gemini API — see section 8) with a structured prompt requesting a fixed JSON output: question text, type (single/multi), answer options, correct-answer flag(s), and a reference to the source passage the question was derived from.
4. Present the draft to the teacher for review and approval (see 3.2) — this step is mandatory and cannot be skipped, to guarantee factual accuracy.

5. Only after teacher approval do the questions become part of the live quiz template.

6.2 Key considerations

- Persist `source_reference` for every question — lets the teacher fact-check quickly and reduces the practical impact of model hallucination.
- Limit supported source types/sizes in phase 1 (public HTML pages + PDF/DOCX up to a defined size), expanding gradually.
- Make explicit to the teacher that AI-generated content may contain errors, and that manual approval is their responsibility.

7. Non-Functional Requirements

Category	Requirement
Platform	Progressive Web App (PWA), mobile-first responsive design, installable to home screen on iOS and Android; single codebase
Performance	Quiz screen loads in under 2 seconds on a standard mobile connection; score computed and shown in under 1 second
Availability	99.5% server uptime during active hours (elevated SLA during live exam windows)
Security	TLS in transit, encryption at rest; personal data (national ID) handled per applicable privacy regulations
Resilience	Answers are cached client-side and retried until confirmed by the server, to tolerate brief connection drops; full offline exam-taking is out of scope for v1
Accessibility	WCAG 2.1 AA baseline (contrast, text size, screen-reader support)
Internationalization	Hebrew (RTL) as the default UI language; architecture must not hard-code LTR assumptions, to allow additional languages later
Scalability	Support several hundred concurrent students during a quiz-opening peak
Exam integrity	Time limit (3.7) is enforced authoritatively server-side, independent of the client. Focus-loss detection (3.8) is best-effort only, via Page Visibility API and blur events — a deterrent, not a guaranteed lockdown; see the design note in 3.8 for the concrete limits of what a PWA can enforce

8. Recommended Technical Architecture

Component	Recommendation	Rationale
Client	React + Vite, PWA (installable, responsive, mobile-first)	Single codebase for student and teacher UIs; installable app-like experience without app-store overhead
Backend / API	Node.js (NestJS or Express) or Python (FastAPI), REST API	Straightforward integration with the Google Generative AI SDK for AI generation
Database	PostgreSQL	Relational, matches the schema in section 5 well, supports indexing for score-query performance
AI generation engine	Google Gemini API	Chosen per owner decision: an existing paid Gemini subscription makes this the lower-marginal-cost option compared to adding a separate AI provider. Gemini supports the structured/JSON output the schema in 5.2

Component	Recommendation	Rationale
		requires, and its native multimodal capability is directly useful for the image/diagram requests in F-10c.
File storage	S3-compatible object storage (source docs, Excel exports)	Standard, cheap, simple to integrate
Auth (teacher)	JWT-based session auth	Industry standard, simple to implement

Design note — keep the provider swappable

Recommend implementing the AI generation engine behind a small internal interface (e.g. `generateQuestions()`, `improveQuestion()`, `reviseQuizFeedback()`) inside the ai-generation module (9.1), rather than calling the Gemini SDK directly from route handlers. This costs almost nothing up front, and means comparing providers on cost or quality later — or falling back to a second provider if Gemini has an outage — is a contained change instead of a rewrite. This is generally good practice regardless of which provider is primary; it's called out here because a cost-driven provider choice is exactly the kind of decision that's worth being able to revisit cheaply.

9. Implementation Appendix (for engineering handoff)

This section is written specifically to seed implementation work in Claude Code. It is illustrative, not final — Claude Code should treat it as a strong starting point and adjust as the codebase takes shape.

9.1 Suggested repository layout

```
bohan-peta/  
  apps/  
    web/                                # React + Vite PWA (student + teacher UI)  
      src/  
        routes/  
          student/                      # join, take-quiz, result screens  
          teacher/                      # quiz builder, cohorts, scores  
        components/  
        lib/api-client.ts  
  api/                                  # backend service  
    src/  
      modules/  
        students/  
        teachers/  
        cohorts/  
        quiz-templates/  
        quiz-assignments/  
        attempts/  
        ai-generation/  
      db/  
        migrations/  
        schema.sql  
  packages/  
    shared-types/                      # TS types shared between web and api  
  docs/  
    BOHAN-PETA_PRD_SRS.docx          # this document
```

9.2 Database schema (PostgreSQL DDL, starting point)

```
CREATE TABLE teacher (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name TEXT NOT NULL,  
  email TEXT UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL,  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

```

CREATE TABLE cohort (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  teacher_id UUID REFERENCES teacher(id),
  name TEXT NOT NULL,
  start_date DATE,
  end_date DATE
);

CREATE TABLE quiz_template (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  teacher_id UUID REFERENCES teacher(id),
  title TEXT NOT NULL,
  language TEXT NOT NULL DEFAULT 'en',
  difficulty TEXT CHECK (difficulty IN ('easy','medium','hard')),
  ai_generated BOOLEAN DEFAULT false,
  status TEXT CHECK (status IN ('draft','published')) NOT NULL DEFAULT 'draft',
  teacher_notes TEXT,
  duration_minutes INT NOT NULL DEFAULT 30,
  pass_score NUMERIC(5,2) NOT NULL DEFAULT 60,
  pass_feedback_text TEXT,
  fail_feedback_text TEXT,
  reveal_answer_key BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT now()
);
-- status toggles Edit Mode <-> Execution Mode (see PRD 3.4).
-- Editing (INSERT/UPDATE on question / answer_option) is allowed
-- regardless of status; status only gates whether students can join.
-- teacher_notes: standing quiz-level AI feedback (PRD 3.6, F-19b).
-- duration_minutes: PRD 3.7, F-04. Required. Enforce server-side by
-- comparing now() to attempt.started_at + duration_minutes, not by
-- trusting the client's own countdown display.
-- pass_score/pass_feedback_text/fail_feedback_text: PRD 3.7, F-27/F-28.
-- reveal_answer_key: PRD 3.1, F-07b. Gates only the FULL answer key on
-- the review link; per-question correct/incorrect status (F-07a) is
-- always shown regardless of this flag.
-- Editable at any time; changes are not retroactive on stored scores.

CREATE TABLE quiz_source (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quiz_template_id UUID REFERENCES quiz_template(id) ON DELETE CASCADE,
  source_type TEXT CHECK (source_type IN ('url','file')) NOT NULL,
  url TEXT,
  storage_key TEXT
);
-- One or more rows per quiz_template (PRD F-10); replaces a single
-- source_document_url field to support multi-source generation.

CREATE TABLE question (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quiz_template_id UUID REFERENCES quiz_template(id) ON DELETE CASCADE,
  text TEXT NOT NULL,
  type TEXT CHECK (type IN ('single','multi')) NOT NULL,
  source_reference TEXT,
  is_active BOOLEAN NOT NULL DEFAULT true,
  teacher_notes TEXT,
  image_url TEXT,
  image_prompt TEXT,
  sort_order INT
);
-- Soft-delete via is_active=false once an AttemptAnswer references
-- this question; hard DELETE is only safe pre-attempt.
-- teacher_notes: standing free-text instructions for the
-- per-question 'Improve with AI' dialog (PRD 3.5).
-- image_url/image_prompt: F-10c. image_prompt records what was
-- requested/generated for audit and re-generation.

CREATE TABLE answer_option (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  question_id UUID REFERENCES question(id) ON DELETE CASCADE,
  text TEXT NOT NULL,
  is_correct BOOLEAN NOT NULL DEFAULT false
);

```



```

CREATE TABLE quiz_assignment (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quiz_template_id UUID REFERENCES quiz_template(id),
  cohort_id UUID REFERENCES cohort(id),
  access_code TEXT UNIQUE NOT NULL,
  open_at TIMESTAMPTZ,
  close_at TIMESTAMPTZ,
  max_attempts INT DEFAULT 1,
  shuffle BOOLEAN DEFAULT true
);

CREATE TABLE student (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  first_name TEXT NOT NULL,
  last_name TEXT NOT NULL,
  national_id TEXT NOT NULL,
  email TEXT
);

CREATE TABLE attempt (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quiz_assignment_id UUID REFERENCES quiz_assignment(id),
  student_id UUID REFERENCES student(id),
  started_at TIMESTAMPTZ DEFAULT now(),
  submitted_at TIMESTAMPTZ,
  score NUMERIC(5,2),
  ended_reason TEXT CHECK (ended_reason IN ('submitted','focus_loss','time_expired')),
  UNIQUE (quiz_assignment_id, student_id) -- enforces single attempt by default
);
-- ended_reason: PRD 3.7/3.8, F-04/F-08/F-08a. 'focus_loss' marks an
-- attempt auto-submitted because the exam screen lost focus;
-- 'time_expired' marks one auto-submitted because duration_minutes
-- elapsed (server-enforced, see quiz_template note above).

CREATE TABLE attempt_answer (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  attempt_id UUID REFERENCES attempt(id) ON DELETE CASCADE,
  question_id UUID REFERENCES question(id),
  selected_option_ids UUID[]
);

```

Note: the UNIQUE(quiz_assignment_id, student_id) constraint enforces the single-attempt default from F-03. When max_attempts > 1, relax this to a non-unique index and enforce the attempt cap in application logic instead.

9.3 REST API surface (starting point)

Method & path	Purpose	Auth
POST /assignments/join	Student registers for an attempt (F-01–F-03); body includes the access token plus student fields — the token is never part of the URL, per the single shared Join Exam screen in 3.1. Returns an attempt id.	Public
GET /attempts/:id/questions	Fetch the full question set for this attempt (shuffled per settings) — client paginates locally, one question per page (F-04a)	Attempt session
PUT /attempts/:id/answers/:questionId	Save/update the student's answer for one question; idempotent, called on every navigation/answer change (F-04b) and used to power resume (F-05)	Attempt session
POST /attempts/:id/submit	Submit answers, compute score, classify pass/fail against pass_score, return the matching feedback message plus a review link (F-06, F-07a, 3.7)	Attempt session

Method & path	Purpose	Auth
POST /attempts/:id/auto-submit	Client-triggered on focus loss (3.8, F-08): submits current answers, sets ended_reason='focus_loss', returns the same result payload (incl. review link) as a normal submit	Attempt session
GET /attempts/:id/review	Per-question correct/incorrect breakdown for this attempt (F-07a); includes correct option text only if the quiz's reveal_answer_key is true (F-07b). :id (Attempt UUID) is itself the bearer credential — see the design note in 3.1	Attempt token (bearer link)
POST /auth/login	Teacher login	Public
POST /quiz-templates	Create a quiz template (manual or as an AI-draft container)	Teacher
POST /quiz-templates/:id/generate	Trigger AI generation (F-10–F-12); body: sources[], language, difficulty, question_count, sub_topics, options_per_question, allow_multi_select, request_images	Teacher
PATCH /quiz-templates/:id/questions/:qid	Edit/approve a draft question, or edit a question on an already-published quiz (F-18)	Teacher
POST /quiz-templates/:id/questions/:qid/improve	Per-question AI dialog (3.5, F-19): body includes free-text notes; returns a proposed revision; also persists notes to teacher_notes (F-19a)	Teacher
POST /quiz-templates/:id/feedback	Quiz-level AI dialog (3.6, F-19b): body includes free-text notes covering the whole quiz; returns a batch of proposed per-question changes; persists notes to quiz_template.teacher_notes	Teacher
POST /quiz-templates/:id/questions/:qid/image	Request an image/diagram for a specific question (F-10c) — tries source-document extraction first, falls back to structured diagram generation per the design note in 3.2	Teacher
PATCH /quiz-templates/:id/status	Toggle between 'draft' (Edit Mode) and 'published' (Execution Mode) (F-18, 3.4)	Teacher
PATCH /quiz-templates/:id/grading	Set duration_minutes, pass_score, pass_feedback_text, fail_feedback_text, reveal_answer_key (F-04, F-27, F-28, F-07b, 3.7)	Teacher
DELETE /quiz-templates/:id/questions/:qid	Remove a question — soft-delete (is_active=false) if it has recorded answers, hard delete otherwise	Teacher
POST /quiz-templates/:id/import	Import questions from JSON or Excel (F-16)	Teacher
GET /quiz-templates/:id/export	Export questions to JSON or Excel (F-16)	Teacher
POST /cohorts	Create a cohort (F-20)	Teacher
POST /cohorts/:id/assignments	Assign a quiz template to a cohort (F-21, F-22)	Teacher
GET /cohorts/:id/scores	Cohort-scoped score sheet (F-23), including pass/fail per row per the quiz's pass_score (F-27)	Teacher
GET /cohorts/:id/scores/export	Export cohort score sheet to Excel/CSV (F-24)	Teacher

9.4 Suggested build order for Claude Code

1. Scaffold repo (apps/web, apps/api, packages/shared-types) and the Postgres schema from 9.2.
2. Implement teacher auth + cohort CRUD (F-20) — smallest vertical slice, no AI dependency.
3. Implement manual quiz-template authoring (F-14, F-13) end-to-end, including quiz-assignment creation (F-21, F-22).
4. Implement the student join → take-quiz → submit → score flow (F-01–F-07) against manually authored quizzes.
5. Implement the cohort-scoped scores view and Excel/CSV export (F-23, F-24).
6. Add AI-assisted generation (F-10–F-12) as a layer on top of the already-working manual authoring flow.
7. Add JSON/Excel import-export for quiz templates (F-16) and question bank reuse (F-17).

9.5 Sample quiz — worked example / seed fixture

A concrete example of a published quiz template in the JSON format defined in 5.2, at the medium-high difficulty level a teacher would expect from an AI generation pass over the Playwright Python docs. Reviewed for technical accuracy (actionability checks, `page.route()`, strict-mode locator behavior, sync tracing API, `APIRequestContext`). Recommended for use directly as `apps/api/db/seed/sample-quiz.json` — a real fixture is more useful for early development and testing than placeholder Lorem-ipsum data.

```
{
  "quiz_title": "Playwright with Python – Core Concepts",
  "language": "en",
  "difficulty": "medium-high",
  "sources": [
    "https://playwright.dev/python/docs/"
  ],
  "duration_minutes": 20,
  "pass_score": 60,
  "pass_feedback_text": "Nice work – you have a solid grasp of Playwright's core mechanics. Move on to the next module.",
  "fail_feedback_text": "You are not quite there yet – review Actionability, Locators, and Network Interception in the docs, then retake this quiz.",
  "reveal_answer_key": false,
  "questions": [
    {
      "text": "Playwright performs a set of \"Actionability\" checks on an element before performing actions like click(). Which of the following is NOT a required actionability check for a standard click() operation?",
      "type": "single",
      "source_reference": "Actionability checks",
      "options": [
        { "text": "The element is Attached to the DOM.", "is_correct": false },
        { "text": "The element is Visible.", "is_correct": false },
        { "text": "The element is Focused.", "is_correct": true },
        { "text": "The element Receives Events (e.g., not obscured by other elements).", "is_correct": false }
      ]
    },
    {
      "text": "What is the correct way to intercept network requests and mock an API response in Playwright (Python)?",
      "type": "single",
      "source_reference": "Network > Mocking APIs",
      "options": [
        { "text": "page.mock_route(\"**/api/data\", handler)", "is_correct": false },
        { "text": "page.route(\"**/api/data\", handler)", "is_correct": true },
        { "text": "page.intercept(\"**/api/data\", handler)", "is_correct": false },
        { "text": "page.network.mock(\"**/api/data\", handler)", "is_correct": false }
      ]
    },
    {
      "text": "By default, what happens if you use page.locator(\".submit-btn\").click() and the locator resolves to multiple elements on the page?",
      "type": "single",
      "source_reference": "Locators > Strict mode",
      "options": [
        { "text": "Playwright automatically clicks the first matching element it finds.", "is_correct": false },

```

```

    {"text": "Playwright clicks all matching elements sequentially.", "is_correct": false},
    {"text": "Playwright throws an Error (strict mode violation) because it requires the locator to resolve to a single element.", "is_correct": true},
    {"text": "Playwright waits until only one element matches the locator, failing if the timeout is reached.", "is_correct": false}
  ],
},
{
  "text": "How do you programmatically start and stop tracing to capture a zip file containing DOM snapshots, screenshots, and network requests using the synchronous API?",
  "type": "single",
  "source_reference": "Trace viewer > Recording a trace",
  "options": [
    {"text": "context.tracing.start(snapshots=True) followed by context.tracing.stop(path=\"trace.zip\")", "is_correct": true},
    {"text": "page.trace.start(screenshots=True) followed by page.trace.stop(file=\"trace.zip\")", "is_correct": false},
    {"text": "browser.start_tracing() followed by browser.export_trace(\"trace.zip\")", "is_correct": false},
    {"text": "context.start_trace(snapshots=True) followed by context.stop_trace(\"trace.zip\")", "is_correct": false}
  ],
},
{
  "text": "When writing tests that require direct HTTP(S) communication without loading a browser window (e.g., for API testing or test data setup), which Playwright class should you use?",
  "type": "single",
  "source_reference": "API testing",
  "options": [
    {"text": "BrowserContext", "is_correct": false},
    {"text": "HTTPRequestContext", "is_correct": false},
    {"text": "APIRequestContext", "is_correct": true},
    {"text": "NetworkInterceptor", "is_correct": false}
  ],
}
]
}

```

Note: source_reference is included per-question here even though it was omitted from the minimal schema shown in 5.2 — it should be treated as part of the real schema, since it maps directly to Question.source_reference in the DB (section 5) and is what lets a teacher fact-check an AI-drafted question against the original doc.

10. Risks & Open Questions

#	Topic	Detail
1	AI question quality	Mandatory manual approval mitigates this; consider a teacher feedback signal to improve the generation prompt over time
2	National ID privacy	Confirm data-handling/consent requirements with legal counsel, given national ID is stored
3	Cheating prevention	Focus-loss auto-submit (3.8) is now specified as a deterrent; it is not equivalent to true lockdown/proctoring. Decide if/when stronger measures (e.g. native lockdown, randomized proctoring) become necessary
4	AI source coverage	Which source types/sites are actually supported in phase 1? Define a closed list for test coverage
5	Reversibility of the PWA decision	If push notifications, true offline exam-taking, or hard exam lockdown (3.8) become a hard requirement, plan for a Capacitor-based native wrapper rather than a rewrite

11. Success Metrics

- Average time to create and publish a new quiz (start to finish) under 10 minutes.

- Share of AI-generated questions that require no substantive teacher edit: above 70%.
- Quiz screen load time for students: under 2 seconds.
- Post-use teacher satisfaction (NPS): above 40.